Aim: To implement logistic regression for binary classification problem and analysis the classification performance

Naman Udapure CS23093

```
import seaborn as sns
import pandas as pd
import numpy as np
```

```
df = sns.load_dataset ('iris')
df.head()
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

Next steps: Generate code with df New interactive sheet

```
df.describe()
```

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal_length  150 non-null    float64
 1   sepal_width   150 non-null    float64
 2   petal_length  150 non-null    float64
 3   petal_width   150 non-null    float64
 4   species       150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

```
df.shape
```

```
(150, 5)
```

```
df
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

150 rows × 5 columns

Next steps: Generate code with df New interactive sheet

```
df['species'].unique()
```

```
array(['setosa', 'versicolor', 'virginica'], dtype=object)
```

```
df.isnull().sum()
```

	0
sepal_length	0
sepal_width	0
petal_length	0
petal_width	0
species	0

dtype: int64

```
df=df[df['species']!='setosa']
```

```
df.head()
```

	sepal_length	sepal_width	petal_length	petal_width	species
50	7.0	3.2	4.7	1.4	versicolor
51	6.4	3.2	4.5	1.5	versicolor
52	6.9	3.1	4.9	1.5	versicolor
53	5.5	2.3	4.0	1.3	versicolor
54	6.5	2.8	4.6	1.5	versicolor

Next steps: Generate code with df New interactive sheet

```
df.shape
```

```
(100, 5)
```

```
df['species']=df['species'].map({'versicolor':0,'virginica':1})
```

```
/tmp/ipython-input-414/2618950047.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a
  df['species']=df['species'].map({'versicolor':0,'virginica':1})
```

```
df.head()
```

	sepal_length	sepal_width	petal_length	petal_width	species
50	7.0	3.2	4.7	1.4	0
51	6.4	3.2	4.5	1.5	0
52	6.9	3.1	4.9	1.5	0
53	5.5	2.3	4.0	1.3	0
54	6.5	2.8	4.6	1.5	0

Next steps: Generate code with df New interactive sheet

```
x=df.iloc[:,:-1]
print(x)
```

```
     sepal_length  sepal_width  petal_length  petal_width
50            7.0          3.2           4.7          1.4
51            6.4          3.2           4.5          1.5
52            6.9          3.1           4.9          1.5
53            5.5          2.3           4.0          1.3
54            6.5          2.8           4.6          1.5
..            ...          ...           ...          ...
145           6.7          3.0           5.2          2.3
146           6.3          2.5           5.0          1.9
147           6.5          3.0           5.2          2.0
148           6.2          3.4           5.4          2.3
149           5.9          3.0           5.1          1.8

[100 rows x 4 columns]
```

```
y=df.iloc[:,-1]
print(y)
```

```
50     0
51     0
52     0
53     0
54     0
      ..
145    1
146    1
147    1
148    1
149    1
Name: species, Length: 100, dtype: int64
```

```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test= train_test_split(x, y, test_size=0.3, random_state=42)
```

```
from sklearn.linear_model import LogisticRegression
classifier=LogisticRegression()
```

```
from sklearn.model_selection import GridSearchCV
parameters={'penalty':['l1','l2'], 'C':[1,2,3,4,5,6,10,20,30,40,50], 'max_iter':[100, 200, 500, 1000]}
```

```
classifier_regressor=GridSearchCV(classifier, param_grid=parameters, scoring='accuracy', cv=5)
```

```
classifier_regressor.fit(x_train, y_train)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_validation.py:528: FitFailedWarning:
220 fits failed out of a total of 440.
The score on these train-test partitions for these parameters will be set to nan.
If these failures are not expected, you can try to debug them by setting error_score='raise'.

Below are more details about the failures:
--------------------------------------------------------------------------------
220 fits failed with the following error:
Traceback (most recent call last):
  File "/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_validation.py", line 866, in _fit_and_score
    estimator.fit(X_train, y_train, **fit_params)
  File "/usr/local/lib/python3.12/dist-packages/sklearn/base.py", line 1389, in wrapper
    return fit_method(estimator, *args, **kwargs)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py", line 1193, in fit
    solver = _check_solver(self.solver, self.penalty, self.dual)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py", line 63, in _check_solver
    raise ValueError(
ValueError: Solver lbfgs supports only 'l2' or None penalties, got l1 penalty.

  warnings.warn(some_fits_failed_message, FitFailedWarning)
/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_search.py:1108: UserWarning: One or more of the test s
        nan 0.97142857        nan 0.97142857        nan 0.97142857
        nan 0.97142857        nan 0.97142857        nan 0.97142857
        nan 0.97142857        nan 0.97142857        nan 0.97142857
        nan 0.97142857        nan 0.97142857        nan 0.97142857
        nan 0.97142857        nan 0.97142857        nan 0.97142857
        nan 0.97142857        nan 0.97142857        nan 0.97142857
        nan 0.97142857        nan 0.97142857        nan 0.97142857
        nan 0.97142857        nan 0.97142857        nan 0.97142857
        nan 0.97142857        nan 0.97142857        nan 0.97142857
        nan 0.97142857        nan 0.97142857        nan 0.97142857
        nan 0.97142857        nan 0.97142857        nan 0.97142857
        nan 0.98571429        nan 0.98571429        nan 0.98571429
        nan 0.98571429        nan 0.98571429        nan 0.98571429
        nan 0.98571429        nan 0.98571429]
  warnings.warn(
```

```
▸ GridSearchCV
▸ best_estimator_: LogisticRegression
  ▸ LogisticRegression
```

```
print(classifier_regressor.best_params_)
```

```
{'C': 40, 'max_iter': 100, 'penalty': 'l2'}
```

```
print(classifier_regressor.best_score_)
```

```
0.9857142857142858
```

```
y_pred=classifier_regressor.predict(x_test)
```

```
score=classifier_regressor.score(x_test, y_test)
print(score)
```

```
0.9333333333333333
```

```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.94      0.94      0.94        17
           1       0.92      0.92      0.92        13

    accuracy                           0.93        30
   macro avg       0.93      0.93      0.93        30
weighted avg       0.93      0.93      0.93        30
```

```
sns.pairplot(df, hue='species')
```

```
<seaborn.axisgrid.PairGrid at 0x7aa2bc2aabd0>
```